

gn

get Notes : A Minimalist Cloud-Synced CLI Note System

Version 1.0.2026

gn-notes.pages.dev · MIT License · Barney Matthews

CONTENTS

1	Philosophy & Architecture
2	How It Works
3	Getting Started
4	Core Configuration Setup
5	Storage Backend: Koofr WebDAV
6	Storage Backend: GitHub API
7	Command Line Reference
8	Local File Commands
9	Advanced Synchronization Layer
10	Markdown Style & Metadata Guide
11	Unix Command Line Integration
12	Mobile Workflows & Integration
13	Troubleshooting & Conflict Resolution
14	Quick Reference Blueprint
15	Complete Script Source Code

1. PHILOSOPHY & ARCHITECTURE

In the modern landscape of personal information management, productivity software has increasingly drifted toward bloat, complex cloud tracking, proprietary file formats, and heavy user interfaces. gn stands as a sharp counter-movement to this complexity.

Built as a single, highly optimized bash script, gn provides a secure bridge between your local workstation terminal and your chosen private cloud storage. It relies on zero heavy local background sync daemons, zero external application libraries, and zero specialized file formats.

Your notes are standard, human-readable Markdown files. They open in any editor, can be manipulated by standard command-line tools like grep, and are entirely within your complete ownership and control.

Design Pillars

The entire functional surface area of gn is governed by four core design pillars:

- **Zero Dependency** — Requires nothing more than standard POSIX utilities (curl, grep, sed, awk) available on any Linux, macOS, or Unix-like ecosystem.
- **Last-Write-Wins** — Rather than maintaining complex multi-master client synchronization graphs, it uses an explicit pull-on-open and push-on-save lifecycle tracking method.
- **Editor Agnostic** — Respects your local workstation choice of environment, launching whichever binary is registered to your `$EDITOR` environment variable.
- **Transparent Privacy** — Credentials stay isolated safely on your local disk within a secured permissions configuration profile file.

Supported Storage Backends

gn currently supports two storage backends, selectable on first run:

- **Koofr WebDAV** — A European privacy-focused cloud provider with native WebDAV protocol support. No plugins required.
- **GitHub API** — Uses the GitHub REST API to store notes as files in a private repository, with full commit history on every save.

2. HOW IT WORKS

Understanding the runtime loop of gn eliminates all ambiguity about how your information flows between your local directory and your secure remote storage server.

Every time you initiate the tool specifying a target note filename, a strict, deterministic workflow executes cleanly from start to finish:

1. The local note management storage folder is initialized at `~/gn/` if it does not already exist on disk.
2. The script dynamically probes your saved credential profile file at `~/gn/gn.conf` to identify your cloud backend choice.
3. It performs an active pull request to the remote storage server to fetch the latest state of the file, overwriting the local copy.

The Local Modification Loop

Once the download phase terminates successfully, or safely notes that a file does not exist yet on the remote server:

1. An initial cryptographic state checksum hash of the file is calculated and cached temporarily in memory.
2. The tool hands control of your terminal over to your system interactive text editor binary.
3. Upon exiting your editor, a secondary state checksum hash is immediately calculated.
4. If the hash remains identical, the synchronization phase is skipped to conserve bandwidth. If a delta is detected, an explicit upload is pushed instantly to your remote server.

■ gn provides no automatic interval saving features. Security and network utilization are prioritized by uploading changes exclusively at the moment of file closure.

Runtime Hash Check

```
PRE_SHA=$(get_file_hash "$NOTE_NAME")
$EDITOR "$NOTE_NAME"
POST_SHA=$(get_file_hash "$NOTE_NAME")

if [ "$PRE_SHA" != "$POST_SHA" ]; then
    push_note "$NOTE_NAME"
fi
```

This simple three-line pattern is the heart of gn's sync model. It is intentionally minimal — no daemons, no watchers, no polling loops.

3. GETTING STARTED

Setting up your workspace to use gn requires less than a minute of preparation. Because it is a portable script, there are no traditional packaging managers or installation binaries to configure.

First, ensure you possess a local binary executable directory. Typically, this points directly to your home path directory under `~/bin/` or `~/local/bin/`.

Run the following sequences of commands within your terminal window to save the executable file and configure your access rights:

```
# Create binary folder if missing
mkdir -p ~/bin

# Move script file to target folder
mv gn.sh ~/bin/gn

# Grant executable execution rights
chmod +x ~/bin/gn
```

Environment Verification

To confirm that your terminal shell can successfully discover your new script installation without requiring absolute paths, query your binary location using your shell built-in commands:

```
which gn
```

If your system returns a clean path reference such as `/home/user/bin/gn`, your environment is correctly tuned.

If no output appears, confirm that your `$PATH` string variable includes your custom binary directory by modifying your shell resource profile configuration file (`.bashrc` or `.zshrc`):

```
export PATH="$HOME/bin:$PATH"
```

Setting Your Text Editor

gn relies entirely on your shell global configurations to establish which software interface to initialize for editing tasks. If you do not explicitly assign an override value, the program defaults to using the basic nano text editor.

To assign a modern, highly optimized text workspace, register your preferred editor inside your environment profile:

```
# For Neovim users
export EDITOR="nvim"

# For Helix users
export EDITOR="hx"

# For Zed editor users
export EDITOR="zed --wait"

# For Visual Studio Code users
export EDITOR="code --wait"

# For Micro editor users
export EDITOR="micro"
```

4. CORE CONFIGURATION SETUP

When you trigger gn for the very first execution pass on a new device, it automatically detects that no credential baseline layer exists on disk yet. Instead of failing or crashing, the system shifts into an interactive step-by-step setup mode.

This mode guides you through selecting your preferred storage platform and entering your connection parameters. Let us review an illustrative example of a first-run initialization routine:

```
$ gn
No config found at ~/gn/gn.conf - let's set one up.
Select your provider:
1) GitHub
2) Koofr
Choice [1-2]:
```

The Layout of gn.conf

Once you complete your interactive interview selections, the configuration state is committed into a simple flat-file variable schema at `~/gn/gn.conf`. The storage schema varies depending on which provider sync engine you activate during setup.

```
# Koofr / WebDAV backend
gn_URL=https://app.koofr.net/dav/Koofr
gn_USER=your_email@domain.com
gn_PASS=xxxx-xxxx-xxxx-xxxx
gn_PATH=/gn

# GitHub backend
GIT_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXXX
GIT_OWNER=your-username
GIT_REPO=notes
```

Security Profiles On Disk

Because your configuration file contains highly critical authentication details and personal account tokens, the script maintains an aggressive local protection model.

Every single time the code initializes, it triggers an explicit system access check command against the target file path, modifying permissions using a strict Unix bitmask:

```
chmod 600 "$CONFIG_FILE"
```


This setting completely restricts read and write visibility, ensuring that no other local system accounts or guest profiles can read or parse the contents of your configuration file.

The config file is also written using `umask 077` at creation time, ensuring secure permissions from the moment it is first saved.

5. STORAGE BACKEND: KOOFR WEBDAV

Koofr is a modern, highly performant European cloud storage provider that offers rich, native compliance with the industry-standard WebDAV network protocol layer out of the box.

Using Koofr with gn provides a robust, seamless file transfer experience that works across all your devices without requiring third-party storage plugins or compiled libraries.

To configure Koofr correctly, never expose your master administrative account password directly within the script environment configurations. Always use an App Password — a scoped, revocable credential that cannot access your account settings or billing.

Generating Application Passwords

Follow these quick operational steps within your Koofr web portal to acquire a safe, scoped access token:

1. Navigate to app.koofr.net and log into your workspace profile.
2. Access your personal profile settings pane by clicking your avatar icon in the upper corner.
3. Select the **Account Settings** module, then open the **Password** configuration tab.
4. Scroll down to the **App Passwords** registration field. Add a descriptive label such as **gn-cli**, then select generate.
5. Copy the resulting 16-character token immediately. It will not be shown again.

Koofr Deployment Variables

PARAMETER KEY	FUNCTIONAL DEFINITION
gn_URL	<code>https://app.koofr.net/dav/Koofr</code>
gn_USER	Your account login email string
gn_PASS	The isolated 16-character app token
gn_PATH	Remote directory name (e.g. <code>/gn</code>)

■ Koofr app passwords are scoped to file operations only. They cannot be used to access account settings, billing, or sharing controls.

6. STORAGE BACKEND: GITHUB API

For developers, engineers, and power users who spend significant time inside terminal consoles, utilizing a secure, private GitHub repository as a cloud storage layer is an exceptional choice.

This engine bypasses standard file transport mechanisms entirely, interacting directly with the official GitHub REST programmatic developer interfaces over encrypted channels.

Your text entries are saved cleanly with built-in history tracking records, providing comprehensive transactional change logs for every file update operation you perform. Each save from gn creates a dated commit message of the form:

```
gn: update project.md 2026-06-18 14:32:07
```

Personal Access Tokens

To authorize transactions securely without disclosing your master login password, generate a scoped Personal Access Token (Classic version):

1. Navigate to github.com/settings/tokens in your browser.
2. Select the configuration control to create a new Classic token.
3. Assign a safe expiration window and check the checkbox scope labeled **repo**.
4. Generate the value and copy the resulting string immediately. Record it into your local setup configuration when prompted by gn.

GitHub Schema Properties

KEY ENTRY	RUNTIME MAPPING ROLE
GIT_TOKEN	Authentication token bearer string
GIT_OWNER	Your individual profile username target
GIT_REPO	The designated repository identifier (e.g. notes)
GIT_API	Automated endpoint destination – auto-resolved if omitted

- Keep the target repository set to Private in GitHub settings. gn works with public repositories but private is strongly recommended for personal notes.

7. COMMAND LINE REFERENCE

The operational syntax governing interactions with gn is built to be simple, predictable, and clean. Running the tool with no parameters initializes a standard default note file named **note.md**.

Any arbitrary text string appended to your command automatically resolves as a target markdown document name. For example, running **gn ideas** loads **ideas.md** from your remote storage.

```
gn [options] [note]
```

Functional Execution Flags

FLAG	RUNTIME ACTION
-h	Displays the built-in reference documentation screen
-d NOTE	Removes a target file from both local and cloud storage
-r OLD NEW	Renames an existing document across both local and remote
-s	Pulls down all remote notes to the local device directory
-c	Wipes your local credentials to reset configuration parameters

Usage Examples

```
gn                # open default note.md
gn project        # open project.md
gn work/standup   # open work/standup.md in a subdirectory
gn -s            # sync all remote notes to local
gn -d draft       # delete draft.md (local + remote)
gn -r old new     # rename old.md to new.md
gn -c            # clear credentials and reconfigure
gn -h            # show help
```

Protected File Filters

To prevent catastrophic accidental data loss or corruption of critical configuration parameters during daily operations, the script features built-in path validation filters.

If you attempt to open or modify an active system file directly via the standard execution flags, the command pipeline immediately halts execution:

```
$ gn gn.conf
# Error: exit 1 – blocked silently

# Paths containing '..' are also blocked:
$ gn ../../etc/passwd
# Error: exit 1
```

This validation ensures your configuration variables remain safe from accidental text replacement or editor formatting errors.

8. LOCAL FILE COMMANDS

Because all notes are plain Markdown files inside `~/gn/`, the entire POSIX toolchain works on them directly. No plugins, no exports, no special modes — just standard shell commands that work identically on macOS, Linux, and NetBSD.

The commands below complement `gn` rather than replace it. Use `gn` for pull/push lifecycle management and these commands for local browsing, searching, and bulk operations.

Listing Notes

```
ls ~/gn/*.md           # list all notes
ls -lt ~/gn/*.md       # newest first by modification time
ls -ls ~/gn/*.md       # largest first by file size
find ~/gn -name "*.md" | sort  # recursive sorted list including subdirs
```

Searching Note Content

```
grep -ril "keyword" ~/gn/  # matching filenames only (case-insensitive)
grep -rn  "keyword" ~/gn/  # matching lines with line numbers
grep -rh  "keyword" ~/gn/  # matching lines, no filenames
grep -rl  "TODO" ~/gn/     # notes containing the word TODO
```

Extract all task lines across every note:

```
grep -rh "^- \[.\]" ~/gn/
```

Open tasks only:

```
grep -rh "^- \[ \]" ~/gn/
```

Completed tasks only:

```
grep -rh "^- \[x\]" ~/gn/
```

Finding Notes by Date or Name

```
find ~/gn -name "*.md" -mtime -1  # modified in the last 24 hours
find ~/gn -name "*.md" -mtime -7  # modified in the last 7 days
find ~/gn -name "2026-*"          # date-stamped notes
find ~/gn -name "*.md" -newer ~/gn/note.md  # newer than a reference file
find ~/gn -name "*.md" -size +10k  # notes larger than 10KB
```

Word Counts & Size

```
wc -l ~/gn/*.md | sort -rn          # largest notes by line count
wc -w ~/gn/*.md | sort -rn          # largest notes by word count
du -sh ~/gn/                        # total size of notes folder
find ~/gn -name "*.md" | wc -l      # total count of notes
```

Quick Appending Without Opening an Editor

When you want to log a quick idea, timestamp, or completed event instantly without initializing a full terminal editor pane, pipe strings directly into files:

```
echo "- [ ] review config $(date)" >> ~/gn/todo.md
echo "## $(date +%F)" >> ~/gn/journal.md
printf "### Meeting notes\n- " >> ~/gn/work.md
cat >> ~/gn/scratch.md              # type freely, Ctrl-D to save
```

■ Files appended this way will push on the next time you open them with `gn`, since the hash will differ from the remote copy.

Backup

```
# Snapshot the entire local notes folder:
cp -a ~/gn "~/gn-backup-$(date +%F)"

# Compressed archive:
tar czf gn-$(date +%F).tar.gz ~/gn/
```

Interactive Picker with `fzf`

For users handling extensive note collections, typing absolute filenames reduces speed. You can chain `gn` with `fzf` to produce an interactive fuzzy-search picker:

```
# Open any note via fuzzy search (requires fzf)
gn $(find ~/gn -name "*.md" -exec basename {} .md \; | fzf)

# Search note contents interactively with fzf + grep preview:
grep -ril "" ~/gn/ | fzf --preview 'cat {}'
```

■ `fzf` is the only optional external tool referenced in this document. All other commands use only POSIX-standard utilities.

9. ADVANCED SYNCHRONIZATION LAYER

The underpinnings of the synchronization layer inside gn rely on strict state checks rather than relying on heavy network data synchronization systems.

This process is managed primarily by checking file properties or using cryptographic hashing routines available natively inside your Unix distribution environment.

By comparing the file state calculated before loading your text editor against the state verified immediately at closure, the system cleanly isolates changes:

```
PRE_SHA=$(get_file_hash "$NOTE_NAME")
$EDITOR "$NOTE_NAME"
POST_SHA=$(get_file_hash "$NOTE_NAME")
```

Cross-Platform Hash Detection

To remain completely portable across different operating systems like Linux, macOS, or BSD systems, the utility runs an automated discovery loop to locate available hashing tools on your system:

```
get_file_hash() {
    if command -v md5sum >/dev/null 2>&1; then
        md5sum "$1" 2>/dev/null | awk '{print $1}'
    elif command -v shasum >/dev/null 2>&1; then
        shasum "$1" 2>/dev/null | awk '{print $1}'
    elif command -v md5 >/dev/null 2>&1; then
        md5 -q "$1" 2>/dev/null || md5 "$1" 2>/dev/null | awk '{print $1}'
    else
        ls -ln "$1" 2>/dev/null | awk '{print $5,$6,$7,$8}'
    fi
}
```

The fallback to `ls -ln` metadata is a last resort for systems with no hashing utilities available. It uses file size and timestamp as a proxy for change detection.

URL Component Encoding

When working with files that include spaces, custom characters, or calendar dates, raw text parameters can easily break network interactions.

To prevent these transaction errors, the engine processes filenames through a native URL encoding loop before launching any network request:

```
urlenc() {
    local s="$1" out="" c i
    for (( i=0; i<${#s}; i++ )); do
        c="${s:$i:1}"
        case "$c" in
            [a-zA-Z0-9./_-]) out+="$c" ;;
            *) out+=$(printf '%%%02X' "$c") ;;
        esac
    done
    echo "$out"
}
```

■ Credential files are written to mktemp paths with chmod 600 and removed immediately after each curl call. Tokens never appear in the process list.

10. MARKDOWN STYLE & METADATA GUIDE

To ensure that notes remain human-readable across alternative reading environments, a structured layout framework is highly advised.

Using consistent structural templates keeps your single-folder system extremely responsive and allows grep text sweeps to extract targeted tasks smoothly.

Recommended Front-Matter Architecture

Instantiating each file with a lightweight text header properties section isolates taxonomy traits neatly from descriptive content logs:

```
---
title: Project Workspace Summary
date: 2026-06-18
tags: [infrastructure, planning]
---
```

```
# Project Workspace Summary
```

```
Notes begin here...
```

Task Marker Rules

When tracking active tasks inside your markdown documents, maintain a strict bracket notation configuration so your command-line filters read status strings cleanly:

- - `[]` represents a raw unstated backlog or pending operational request item.
- - `[-]` describes an actively running project pipeline currently under revision.
- - `[x]` maps to a finalized operational transaction step completely wrapped up.

By enforcing this standard uniformly, text parser utilities can separate open obligations from finalized historical transactions in milliseconds:

```
grep -rh "^- \[ \]" ~/gn/      # all open tasks
grep -rh "^- \[-\]" ~/gn/     # all in-progress
grep -rh "^- \[x\]" ~/gn/     # all completed
```

Text Layout Parameters

SYNTAX RULE	STRUCTURAL LAYOUT ASSIGNMENT
Headers	Restrict files to a single level-1 main heading component
Lists	Deploy single hyphens exclusively for basic bullet sequences
Tables	Utilize simple text pipe grids sparingly to mitigate visual clutter
Line length	Wrap at 80 characters for comfortable terminal readability
Filenames	Use lowercase hyphenated names for easy tab-completion: project-notes.md

11. UNIX COMMAND LINE INTEGRATION

Because your entire repository collection scales as flat, standard text documents inside `~/gn/`, the environment natively integrates with existing POSIX core tools.

You can instantly build powerful search loops, context filters, and navigation routines right from your native shell window without complex plug-ins.

Harnessing grep for Deep Text Inquiries

To sweep through all note logs searching for context matches, run standard recursive match filters over your target folder:

```
# Search notes for specific project labels case-insensitively
grep -ri "infrastructure" ~/gn/*.md

# Show only filenames that match
grep -ril "infrastructure" ~/gn/

# Show line numbers alongside matches
grep -rn "infrastructure" ~/gn/
```

Fuzzy File Finding via fzf

For users handling extensive text logs, typing absolute filenames reduces speed. You can daisy-chain gn with fzf to produce an interactive picker:

```
# Interactive note selector
gn $(find ~/gn -name "*.md" -exec basename {} .md \; | fzf)

# Search note contents with preview
grep -ril "" ~/gn/ | fzf --preview 'cat {}'
```

Executing this profile shortcut lets you search note titles dynamically with auto-updating list selections as you type.

Bypassing the Text Editor Screen

When you want to log a quick idea, timestamp, or completed event instantly without initializing a full terminal editor pane, pipe strings directly into files:

```
# Append quick line logs on the fly
echo "- [ ] verify cloud backups $(date)" >> ~/gn/todo.md

# Append a dated section header
echo "## $(date +%F)" >> ~/gn/journal.md

# Stream stdin directly into a note
cat >> ~/gn/scratch.md
```

This updates your tracking ledger instantly and keeps local files ready to sync cleanly the next time you trigger a pull sequence loop.

12. MOBILE WORKFLOWS & INTEGRATION

Even though `gn` is primarily optimized as a fast command-line terminal environment utility, your note collections remain highly accessible across mobile platforms like iOS and Android.

Because your files are saved into standard, non-proprietary cloud directories as plain Markdown text documents, any mobile application with direct folder access can connect directly to them.

This architectural choice eliminates the synchronization lock-in and monthly platform tracking fees associated with modern commercial note-taking applications.

Recommended Mobile Software

APPLICATION	PLATFORM	BEST SUITED FOR
iA Writer	iOS / macOS	Clean, minimalist writing with native cloud folder links
Markor	Android	Open-source markdown utility featuring direct folder editing
1Writer	iOS	Extensive action script customization and deep WebDAV links
Obsidian	iOS / Android	Graph view, backlinks, and plugin ecosystem
Working Copy	iOS	Git client – direct sync with GitHub-backed <code>gn</code> repos

iOS Automation Shortcuts

To access your scratchpad instantly with a single tap on your mobile home screen, you can construct an automated system shortcut workflow following these core design steps:

1. Initialize the system date utility action, formatting the output pattern string as: `YYYY-MM-DD`.
2. Assign a string variable named `filename` to group text elements: `Text-LOG.md`.
3. Trigger your local file provider utility action to open the document directly inside your mobile editor application.

■ Users with GitHub-backed `gn` repos can use the Working Copy iOS app to pull, edit, and push notes directly – the full `gn` workflow from a phone.

13. TROUBLESHOOTING & CONFLICT RESOLUTION

Because gn is engineered around a clean, transparent code structure, resolving operational issues or identifying connectivity failures is straightforward.

Most common runtime errors point directly to mismatched configuration credentials, strict corporate firewall profiles, or basic network connection drops.

Common Diagnostics

SYMPTOM	RESOLUTION
Pull failed (HTTP 404)	Normal for new notes. Save and exit to create the remote file automatically.
Push failed (HTTP 409 / 422)	SHA mismatch on GitHub. Run <code>gn -s</code> to pull latest state, then retry.
Push failed (HTTP 401)	Token expired or incorrect. Run <code>gn -c</code> to reconfigure credentials.
<code>curl: (6) Could not resolve host</code>	Check network connectivity. Confirm <code>gn_URL</code> or <code>GIT_API</code> value in <code>gn.conf</code> .
Permission denied on <code>gn.conf</code>	Run: <code>chmod 600 ~/gn/gn.conf</code>
Editor opens empty on every run	Remote pull returned 404 – note does not yet exist on remote. Normal.
No changes. Sync skipped.	File was not modified. Expected behaviour – no action required.
<code>base64: invalid input</code>	Corrupt remote file encoding. Delete remote file and push again.

Managing Simultaneous Updates

Because the engine utilizes a direct last-write-wins method to maximize speed, editing the same note concurrently across multiple devices without syncing can cause later updates to overwrite earlier ones.

To avoid overwriting changes when working from multiple devices, execute an explicit data fetch request before initializing any local editing panels:

```
gn -s    # pull all remote notes before starting a new session
```

- For a lightweight conflict audit on GitHub, compare commit timestamps in the repository's commit history before pushing from an older local copy.

14. QUICK REFERENCE BLUEPRINT

Keep this technical execution summary guide handy within your active reference files or pinned directly to your physical workspace monitor frame for quick navigation.

Primary Command Syntaxes

COMMAND EXAMPLE	FUNCTIONAL OPERATIONAL RESULT
<code>gn</code>	Pulls and launches the default scratchpad file (note.md)
<code>gn project</code>	Pulls and launches a specific file named project.md
<code>gn work/standup</code>	Pulls and launches work/standup.md in a subdirectory
<code>gn -s</code>	Forces a complete download sweep of all remote files
<code>gn -d draft</code>	Deletes draft.md from local disk and remote storage
<code>gn -r old new</code>	Renames old.md to new.md across local and remote
<code>gn -c</code>	Wipes saved credentials and triggers reconfiguration wizard
<code>gn -h</code>	Displays the built-in help and local command reference

Local File Quick Reference

COMMAND	RESULT
<code>ls -lt ~/gn/*.md</code>	List notes, newest first
<code>ls -ls ~/gn/*.md</code>	List notes, largest first
<code>grep -ril "term" ~/gn/</code>	Search notes (filenames only)
<code>grep -rn "term" ~/gn/</code>	Search notes (matching lines)
<code>grep -rh "^- \[\]" ~/gn/</code>	All open tasks across notes
<code>grep -rh "^- \[x\]" ~/gn/</code>	All completed tasks
<code>find ~/gn -name "*.md" -mtime -1</code>	Modified today
<code>find ~/gn -name "*.md" -mtime -7</code>	Modified this week
<code>wc -l ~/gn/*.md sort -rn</code>	Largest notes by line count

<code>du -sh ~/gn/</code>	Total notes folder size
<code>cp -a ~/gn ~/gn-backup-\$(date +%F)</code>	Snapshot backup

Internal File Directories

LOCAL DIRECTORY PATH	SYSTEM ROLE AND PURPOSE
~/gn/	Primary workspace containing your markdown document collection
~/gn/gn.conf	Secured authorization profile containing active account parameters
~/bin/gn	The core portable shell execution binary script

gn · Plain text, zero dependencies · 2026

15. COMPLETE SCRIPT SOURCE CODE

For complete transparency, platform auditing, or immediate copy-paste deployment, the complete, unmodified production shell script of gn is included below. The canonical source is always available at gn-notes.pages.dev.

```
#!/usr/bin/env bash
# gn - Get Notes
# A zero-dependency CLI tool to sync markdown notes via GitHub or Koofr.
# Web: gn-notes.pages.dev. Author: Barney Matthews. License: MIT

NOTES_DIR="$HOME/gn"
CONFIG_FILE="$NOTES_DIR/gn.conf"

mkdir -p "$NOTES_DIR"

for cmd in curl grep sed awk tr; do
    command -v "$cmd" >/dev/null 2>&1 || { echo "Error: '$cmd' is required but not installed." >&2; exit 1; }
done
```

```
# --- Security & Resource Cleanup Trap ---
cleanup() {
    rm -f "$nf" "$hdr" "$pfile" "$file.tmp" 2>/dev/null
}
trap cleanup EXIT INT TERM HUP

# --- Config ---
if [ -f "$CONFIG_FILE" ]; then
    chmod 600 "$CONFIG_FILE"
    while IFS='=' read -r key value; do
        key=$(echo "$key" | tr -d ' ')
        [[ -z "$key" || "$key" =~ ^# ]] && continue
```

```

value="${value%%#*}"
value=$(echo "$value" | sed -e 's/^[[:space:]]*//' -e 's/[[:space:]]*$//')
[[ "$value" =~ ^[\'\\"](.*)[\'\\"]$ ]] && value="${BASH_REMATCH[1]}"
case "$key" in
    gn_USER)          gn_USER="$value" ;;
    gn_PASS)          gn_PASS="$value" ;;
    gn_PATH)          gn_PATH="$value" ;;
    gn_URL)           gn_URL="$value" ;;
    GIT_TOKEN)        GIT_TOKEN="$value" ;;
    GIT_OWNER)        GIT_OWNER="$value" ;;
    GIT_REPO)         GIT_REPO="$value" ;;
    GIT_API)          GIT_API="$value" ;;

```

```
        esac
    done < "$CONFIG_FILE"
fi

# --- Interactive First-Run Setup Wizard ---
if [ -z "$gn_USER" ] && [ -z "$GIT_TOKEN" ]; then
    echo "No config found at $CONFIG_FILE - let's set one up."
    echo "Select your provider:"
    echo "1) GitHub"
    echo "2) Koofr"
    printf "Choice [1-2]: "
    read -r provider_choice
```

```
case "$provider_choice" in
  1)
    printf "GitHub Personal Access Token (input hidden): "
    stty -echo; read -r GIT_TOKEN; stty echo; echo
    printf "GitHub username (repo owner): "
    read -r GIT_OWNER
    printf "Repository name: "
    read -r GIT_REPO
    ;;
  *)
    gn_URL="https://app.koofr.net/dav/Koofr"
    printf "Koofr email/username: "
    read -r gn_USER
```

```

        printf "Koofr app password (input hidden): "
        stty -echo; read -r gn_PASS; stty echo; echo
        printf "Remote notes folder [/gn]: "
        read -r gn_PATH
        gn_PATH="${gn_PATH:~/gn}"
        ;;
    esac

    printf "Save this config for future runs? [Y/n] "
    read -r save
    case "$save" in
        [Nn]|[Nn][Oo]) echo "Using credentials for this session only." ;;
        *)

```



```

        umask 077
    {
        if [ -n "$GIT_TOKEN" ]; then
            printf 'GIT_TOKEN=%s\nGIT_OWNER=%s\nGIT_REPO=%s\n' "$GIT_TOKEN" "$GIT_OWNER" "$GIT_REPO"
        else
            printf 'gn_URL=%s\ngn_USER=%s\ngn_PASS=%s\ngn_PATH=%s\n' "$gn_URL" "$gn_USER" "$gn_PASS"
        fi
    } > "$CONFIG_FILE"
    chmod 600 "$CONFIG_FILE"
    echo "Saved to $CONFIG_FILE"
    ;;
esac
fi

```

```

# --- Detect Sync Engine ---
SYNC_ENGINE=""
if [ -n "$GIT_TOKEN" ] && [ -n "$GIT_OWNER" ] && [ -n "$GIT_REPO" ]; then
    SYNC_ENGINE="GITHUB"
    GIT_API="${GIT_API:-https://api.github.com/repos/${GIT_OWNER}/${GIT_REPO}/contents}"
elif [ -n "$gn_USER" ] && [ -n "$gn_PASS" ] && [ -n "$gn_URL" ]; then
    SYNC_ENGINE="KOOFR"
    gn_URL="${gn_URL%/*}"
    gn_PATH="${gn_PATH:-/notes}"
    gn_PATH="/${gn_PATH#/*}"
    [ "$gn_PATH" = "/" ] && gn_PATH="/"
else

```

```
    echo "Error: gn.conf is incomplete. Provide GitHub or Koofr credentials." >&2
    exit 1
fi

EDITOR="${EDITOR:-nano}"

show_help() {
    local remote_info
    case "$SYNC_ENGINE" in
        GITHUB) remote_info="GitHub: ${GIT_OWNER}/${GIT_REPO}" ;;
        *)      remote_info="Koofr: ${gn_URL}${gn_PATH}" ;;
    esac
    cat <<EOF
```

Usage: gn [options] [note]

- h Show this help
- d NOTE Delete a note (local + remote)
- r OLD NEW Rename a note (local + remote)
- s Sync (pull) all remote notes down to local directory
- c Clear saved credentials and reconfigure

Local commands:

- ls -lt \$NOTES_DIR/*.md List notes, newest first
- grep -ril "term" \$NOTES_DIR/ Search notes (filenames only)
- grep -rn "term" \$NOTES_DIR/ Search notes (matching lines)
- find \$NOTES_DIR -name "*.md" -mtime -7 Modified last 7 days

```
wc -l $NOTES_DIR/*.md | sort -rn          Largest notes by lines
```

```
Engine: $SYNC_ENGINE
```

```
Remote: $remote_info
```

```
Local:  $NOTES_DIR
```

```
EOF
```

```
    exit 0
```

```
}
```

```
api_curl() {
```

```
    local hdr rc
```

```
    if [ "$SYNC_ENGINE" = "KOOFR" ]; then
```

```
        local host
```

```

    host=$(echo "$gn_URL" | awk -F/ '{print $3}')
    nf=$(mktemp); chmod 600 "$nf"
    printf 'machine %s\nlogin %s\npassword %s\n' "$host" "$gn_USER" "$gn_PASS" > "$nf"
    curl -s --netrc-file "$nf" "$@"; rc=$?
    rm -f "$nf"; return $rc
else
    hdr=$(mktemp); chmod 600 "$hdr"
    printf -- '-H "Authorization: Bearer %s\n' "$GIT_TOKEN" > "$hdr"
    curl -s -K "$hdr" "$@"; rc=$?
    rm -f "$hdr"; return $rc
fi
}

```



```
echo "$1" | awk '{
    gsub(/\+/, " ");
    while (match($0, /[0-9a-fA-F]{2}/)) {
        hex = substr($0, RSTART+1, 2);
        dec = 0;
        for (i=1; i<=2; i++) {
            c = substr(hex, i, 1);
            if (c ~ /[A-F]/) { dec = dec * 16 + (index("ABCDEF", c) + 9) }
            else if (c ~ /[a-f]/) { dec = dec * 16 + (index("abcdef", c) + 9) }
            else { dec = dec * 16 + c }
        }
        printf "%s%c", substr($0, 1, RSTART-1), dec;
        $0 = substr($0, RSTART+RLENGTH);
    }
}
```



```

    }
    print $0;
}'
}

get_file_hash() {
    if command -v md5sum >/dev/null 2>&1; then md5sum "$1" 2>/dev/null | awk '{print $1}'
    elif command -v shasum >/dev/null 2>&1; then shasum "$1" 2>/dev/null | awk '{print $1}'
    elif command -v md5 >/dev/null 2>&1; then md5 -q "$1" 2>/dev/null || md5 "$1" 2>/dev/null | awk '{print $1}'
    else ls -ln "$1" 2>/dev/null | awk '{print $5,$6,$7,$8}'
    fi
}

```

```

# --- Pure POSIX Engines for Base64 and JSON ---

# Stateful micro-JSON parser (Handles arbitrary formatting and spaces)
posix_parse_json() {
    local target_key="$1"
    awk -v target="$target_key" '
        BEGIN { RS="[{}:,\t\n\"]" }
        $0 == target { found = 1; next }
        found && $0 != "" { print $0; exit }
    '
}

# Native Awk Base64 Encoder

```



```

awk '
BEGIN {
    str = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    for(i=1; i<=64; i++) alphabet[substr(str, i, 1)] = i - 1;
    ORS=""; BINMODE=1;
}
{
    gsub(/[^A-Za-z0-9\+\=\]/, "");
    len = length($0);
    for(i=1; i<=len; i+=4) {
        b1 = alphabet[substr($0, i, 1)];
        b2 = alphabet[substr($0, i+1, 1)];
        b3 = substr($0, i+2, 1);
    }
}

```

```
b4 = substr($0, i+3, 1);

c1 = lshift_posix(b1, 2) + int(b2 / 16);
print sprintf("%c", c1);

if (b3 != "=" && b3 != "") {
    v3 = alphabet[b3];
    c2 = lshift_posix(and_posix(b2, 15), 4) + int(v3 / 4);
    print sprintf("%c", c2);

    if (b4 != "=" && b4 != "") {
        v4 = alphabet[b4];
        c3 = lshift_posix(and_posix(v3, 3), 6) + v4;
```



```

        function lshift_posix(v, n) { while(n-- > 0) v *= 2; return v; }
    ,
}

remote_url() { echo "${gn_URL}$(urlenc "${gn_PATH}/${1}")"; }

remote_mkdir() {
    local dir="$1" path="" part parts target
    if [ -z "$dir" ] || [ "$dir" = "." ]; then target="$gn_PATH"; else target="$gn_PATH/$dir"; fi
    IFS="/" read -ra parts <<< "$target"
    for part in "${parts[@]}"; do
        [ -z "$part" ] && continue
        path="$path/$part"
    done
}

```

```

        [ "$SYNC_ENGINE" = "KOOFR" ] && api_curl -X MKCOL "${gn_URL}${urlenc "$path")" -o /dev/null
    done
}

pull_note() {
    local file="$1"
    if [ "$SYNC_ENGINE" = "GITHUB" ]; then
        local url resp http_code content REPLY
        url="${GIT_API}/${file}"
        resp=$(api_curl -w "\n%{http_code}" "$url")
        http_code="${resp##*'\n'}"
        REPLY="${resp%'\n'*}"
        [ "$http_code" = "404" ] && return 0
    fi
}

```

```

[ "$http_code" != "200" ] && { echo "Error: Pull failed (HTTP $http_code)" >&2; exit 1; }

# FIX: Drop sed. Use stateful POSIX awk tokenizing logic.
content=$(echo "$REPLY" | posix_parse_json "content" | tr -d '\n\r ')

if [ -n "$content" ] && [ "$content" != "null" ]; then
    # FIX: Platform-safe purely native Awk base64 processing
    echo "$content" | posix_b64decode > "$file"
fi
else
    local url http_code
    url=$(remote_url "$file")
    http_code=$(api_curl -w "%{http_code}" -o "$file.tmp" "$url")

```

```

        case "$http_code" in
            200) mv "$file.tmp" "$file" ;;
            404) rm -f "$file.tmp" "$file" ;;
            *)   rm -f "$file.tmp"; echo "Error: Pull failed." >&2; exit 1 ;;
        esac
    fi
}

push_note() {
    local file="$1"
    if [ "$SYNC_ENGINE" = "GITHUB" ]; then
        local url sha sha_resp http_code content msg pfile REPLY safe_msg safe_file
        msg="gn: update $file $(date '+%Y-%m-%d %H:%M:%S')"

```

```
safe_msg=$(printf '%s' "$msg" | sed 's/\\/\\\\/g; s/"/\\"/g')
safe_file=$(printf '%s' "$file" | sed 's/\\/\\\\/g; s/"/\\"/g')
url="${GIT_API}/${safe_file}"

# FIX: Pure POSIX native base64 text-packing step
content=$(posix_b64encode < "$file")

pfile=$(mktemp); chmod 600 "$pfile"
sha_resp=$(api_curl -s "$url")

# FIX: Token-parse JSON object layout properly
sha=$(echo "$sha_resp" | posix_parse_json "sha")
```

```

if [ -n "$sha" ] && [ "$sha" != "null" ]; then
    printf '{"message": "%s", "content": "%s", "sha": "%s"}' "$safe_msg" "$content" "$sha" > "$pfile"
else
    printf '{"message": "%s", "content": "%s"}' "$safe_msg" "$content" > "$pfile"
fi
resp=$(api_curl -w "\n%{http_code}" -X PUT -H "Content-Type: application/json" --data-binary "@$pfile"
rm -f "$pfile"
http_code="${resp##*$'\n'}"
REPLY="${resp%$'\n'}"
[[ "$http_code" =~ ^(200|201)$ ]] || { echo "Error: Push failed (HTTP $http_code)" >&2; exit 1; }
else
    local url http_code dir
    dir=$(dirname "$file")

```

```

        remote_mkdir "$dir"
        url=$(remote_url "$file")
        http_code=$(api_curl -w "%{http_code}" -o /dev/null -X PUT --data-binary "@$file" "$url")
        case "$http_code" in
            200|201|204) ;;
            *) echo "Error: Push failed." >&2; exit 1 ;;
        esac
    fi
}

remote_delete() {
    local file="$1" msg="${2:-gn: delete $1}"
    if [ "$SYNC_ENGINE" = "GITHUB" ]; then

```

```

local url sha sha_resp http_code resp pfile
url="${GIT_API}/${file}"
pfile=$(mktemp); chmod 600 "$pfile"
sha_resp=$(api_curl -s "$url")

# FIX: Parse with the tokenizer
sha=$(echo "$sha_resp" | posix_parse_json "sha")

if [ -z "$sha" ] || [ "$sha" = "null" ]; then rm -f "$pfile"; return 0; fi
printf '{"message":"%s","sha":"%s"}' "$msg" "$sha" > "$pfile"
resp=$(api_curl -w "\n%{http_code}" -X DELETE -H "Content-Type: application/json" --data-binary "@$pfile"
rm -f "$pfile"
else

```



```

        local url; url=$(remote_url "$file")
        api_curl -s -X DELETE "$url" > /dev/null
    fi
}

remote_rename() {
    local old="$1" new="$2"
    if [ "$SYNC_ENGINE" = "GITHUB" ]; then
        cp "$NOTES_DIR/$old" "$NOTES_DIR/$new"
        (cd "$NOTES_DIR" && push_note "$new")
        remote_delete "$old" "gn: rename $old to $new"
        rm -f "$NOTES_DIR/$new"
    else

```

```

        local src dst; src=$(remote_url "$old"); dst=$(remote_url "$new")
        remote_mkdir "$(dirname "$new")"
        api_curl -s -X MOVE -H "Destination: $dst" -H "Overwrite: F" "$src" > /dev/null
    fi
}

delete_note() {
    local file="$1"
    case "$file" in *.md) ;; *) file="{file}.md" ;; esac
    [ -f "$NOTES_DIR/$file" ] || { echo "Error: '$file' not found."; exit 1; }
    printf "Delete '%s'? [y/N] " "$file"
    read -r confirm
    case "$confirm" in [Yy]*) ;; *) echo "Aborted."; exit 0 ;; esac
}

```

```

    remote_delete "$file"
    rm "$NOTES_DIR/$file"
    echo "Deleted."
    exit 0
}

rename_note() {
    local old="$1" new="$2"
    case "$old" in *.md) ;; *) old="{old}.md" ;; esac
    case "$new" in *.md) ;; *) new="{new}.md" ;; esac
    [ -f "$NOTES_DIR/$old" ] || { echo "Error: '$old' not found."; exit 1; }
    remote_rename "$old" "$new"
    mkdir -p "$NOTES_DIR/${dirname "$new"}" 2>/dev/null
}

```

```

    mv "$NOTES_DIR/$old" "$NOTES_DIR/$new"
    echo "Renamed."
    exit 0
}

sync_all_remote() {
    echo "Syncing paths... [$SYNC_ENGINE]"
    if [ "$SYNC_ENGINE" = "GITHUB" ]; then
        local resp
        resp=$(api_curl -s "${GIT_API}")

        # Extract file names robustly using structural symbols as separators
        echo "$resp" | awk '

```

```

BEGIN { RS="[{}," ] }
/\\"name\\":/ {
    gsub(/.*\\"name\\":\\"|\\"/, "");
    if ($0 ~ /\\".md$/) print $0;
}' | while read -r path; do
    [ -n "$path" ] && (cd "$NOTES_DIR" && pull_note "$path")
done
else
    local base_encoded_path xml_response
    base_encoded_path=$(urlenc "$gn_PATH")
    xml_response=$(api_curl -X PROPFIND -H "Depth: infinity" -H "Content-Type: text/xml" "${gn_URL}${base_encoded_path}")
    echo "$xml_response" | grep -oE '<[A-Za-z:]*href>[^<]+' | sed -E 's|<[^>]+>||g' | while read -r path; do
        local dec_path; dec_path=$(urldec "$path")
    done

```

```

        if [[ "$dec_path" =~ .*"$gn_PATH"/(*\md)$ ]]; then
            local rel_path="${BASH_REMATCH[1]}"
            mkdir -p "$NOTES_DIR/${dirname "$rel_path"}" 2>/dev/null
            (cd "$NOTES_DIR" && pull_note "$rel_path")
        fi
    done
fi
echo "Sync complete."
exit 0
}

reconfigure() { rm -f "$CONFIG_FILE"; echo "Config cleared."; exit 0; }

```

```

# --- Execution Entry ---
if [ "$1" = "-r" ]; then rename_note "$2" "$3"; fi
ACTION_RUN=0
while getopts "hcd:s" opt; do
    case $opt in
        h) show_help ;;
        c) ACTION_RUN=1; reconfigure ;;
        d) ACTION_RUN=1; delete_note "$OPTARG" ;;
        s) ACTION_RUN=1; sync_all_remote ;;
        *) show_help ;;
    esac
done
[ "$ACTION_RUN" -eq 1 ] && exit 0

```

```
shift $((OPTIND - 1))

NOTE_NAME="${1:-note}"
if [[ "$NOTE_NAME" == *.* ]] || [ "$NOTE_NAME" = "gn.conf" ]; then exit 1; fi
case "$NOTE_NAME" in *.md) ;; *) NOTE_NAME="${NOTE_NAME}.md" ;; esac

cd "$NOTES_DIR" || exit 1
mkdir -p "$(dirname "$NOTE_NAME")" 2>/dev/null

pull_note "$NOTE_NAME"
PRE_SHA=$(get_file_hash "$NOTE_NAME")
$EDITOR "$NOTE_NAME"
```



```
[ -f "$NOTE_NAME" ] || exit 0
POST_SHA=$(get_file_hash "$NOTE_NAME")
if [ "$PRE_SHA" != "$POST_SHA" ]; then
    push_note "$NOTE_NAME"
fi
```

gn · Plain text, zero dependencies · 2026